

Week 5: Inference – Belief Propagation in Cluster Graphs

Practical Assignment

In last week's practical, we looked at efficient inference for models that can be represented as factor trees. For this week's practical, we revisit two models from Week 3 that *cannot* be represented as factor trees: the Hamming (7,4) error-correction code and the image denoising problem. We will implement the belief propagation (BP) inference algorithm for the following types of cluster graphs (CGs):

- Junction trees (JTs)
- Loopy CGs

For the Hamming (7,4) error-correction code, we will implement BP manually. For both the Hamming (7,4) error-correction code and the image denoising problem, we will also use `emdw`'s built-in functionality to do BP for us.

1 The Hamming (7,4) error-correction code revisited

This question revisits the Hamming (7,4) error-correction code that we used in Question 1 of Week 3's practical. In this question, you should use and build on the work you did for Question 1 of Week 3's practical.

This question assumes that the factors of the models have been defined, and that the appropriate factors have been observed and reduced with the available evidence (i.e., the observed received bits, $r_0, \dots, r_6$). The reduced factors therefore only contain the transmitted bit RVs, $b_0, \dots, b_6$.

1.1 Manual belief propagation (BP) for a junction tree (JT)

The goal of this question is to construct a junction tree (JT) for the Hamming (7,4) error-correction code and implement belief propagation (BP) for the JT manually (i.e., using `emdw`'s factor operations).

- (On paper) Determine the JT for this problem, using the JT construction procedure outlined in our prescribed resources, and using the following variable elimination ordering: $b_4, b_5, b_6, b_1, b_2, b_3, b_0$. Your JT should look something like that in Figure 1. Assign each of the factors of the model to one of the clusters in the JT.
- (On paper) Indicate all the messages required for BP in the JT (there should be 6 of them). Now determine an appropriate message-passing schedule (similar to what you have done for the factor trees in Week 4's Question 1.2(b) and Question 2.1(b)).
- (In code) Construct all the factors of the model in `emdw` and observe and reduce the appropriate factors with the given evidence (use the same sequence of received bits, $r_0, \dots, r_6$, that you used in Week 3). Calculate the cluster potentials according to the assignment of factors to clusters that you have chosen in part (a).
- (In code) Calculate all the BP messages in the JT, using `emdw`'s factor operations, and following the message-passing schedule you determined in point (b).
- (In code) Select one of the data bits, $b_i \in \{b_0, b_1, b_2, b_3\}$, and calculate the beliefs of two clusters and one sepset that contain this RV, using the messages and cluster potentials. For each of these cluster and sepset beliefs, determine the marginal belief over b_i , and then compare these marginal beliefs. Can you see that the JT is *calibrated*?

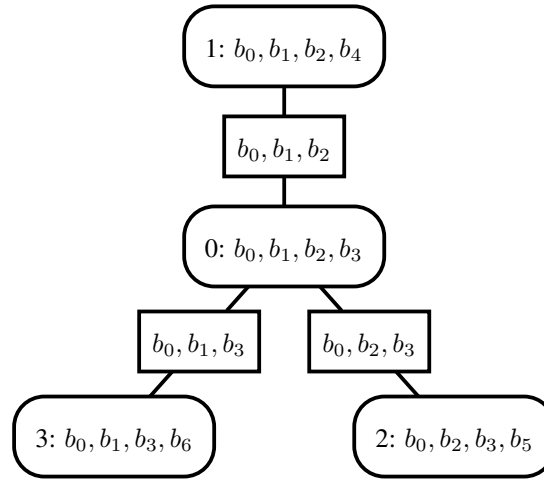


Figure 1: The JT for the Hamming (7,4) error-correction code

- (f) (In code) Calculate the marginal beliefs over each of the transmitted bits, $b_0, \dots, b_6$ (using appropriate sepset or cluster beliefs). Remember that – differently from general factors – beliefs must be valid distributions; so be sure to normalise your calculated beliefs. Compare these beliefs to that obtained from the full joint distribution in Week 3. Is BP in a JT therefore *exact* or *approximate* inference?
- (g) (On paper) Determine the size of the largest table produced during the message-passing process. Compare this with the size of the joint distribution created in Week 3. Can you see the advantage of using a JT compared to the approach of Week 3?

1.2 Manual belief propagation (BP) for a loopy cluster graph (CG)

The goal of this question is to construct a loopy CG for the Hamming (7,4) error-correction code and implement loopy belief propagation (LBP) manually.

- (a) (On paper) Construct a loopy CG for this model. Start by defining a cluster for each of the parity check factors (more precisely: for each parity check factor, choose a cluster with a scope that is the same as that of the parity check factor). Can you see that a CG with these 3 clusters would satisfy the family preservation property? Define the cluster potentials for these 3 clusters by assigning each factor of the model to one of the clusters. Now connect the clusters such that the resulting graph obeys the running intersection property, and is therefore a valid CG. One way to do this is to construct a clique graph (i.e., connect every pair of clusters that share RVs with a sepset that is the intersection of the scopes of the two clusters), and then systematically remove RVs from sepsets until the graph obeys the running intersection property. One possible loopy CG is shown in Figure 2. Here, clusters 1 and 2 share both RVs b_0 and b_3 , but b_0 is removed from the edge

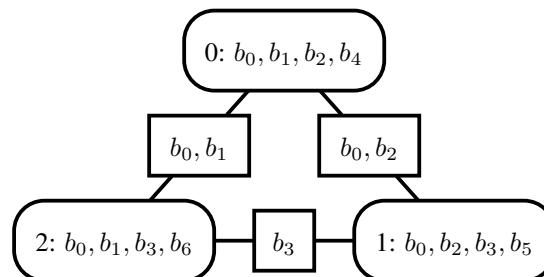


Figure 2: One possible loopy CG for the Hamming (7,4) error-correction code

between clusters 1 and 2 to ensure that there is only one path from cluster 2 to cluster 1 that contains b_0 in all its sepsets (i.e., path 2–0–1). Alternatively, you could have removed b_0 from any of the other two sepsets.

- (b) (On paper) Indicate all the messages required for BP in the CG (there should be 6 of them). Now determine a message-passing schedule. A simple schedule is to send messages in one direction around the loop, and then send messages in the opposite direction around the loop; this schedule should then be repeated until messages converge (this is the so-called *round-robin* schedule).
- (c) (In code) Construct all the factors of the model in `emdw` and observe and reduce the factors using your choice of received bits, $r_0, \dots, r_6$ (similarly to Question 1.1(c)). Now calculate the cluster potentials according to your choice of factor assignment in point (a).
- (d) (In code) Declare and initialise all the messages in the CG. A simple initialisation scheme is to set all the messages to be uniform/uninformative (i.e., all the entries in the table have the same positive value; the exact value of the entries is not important).
- (e) (In code) Calculate all the messages using `emdw`'s factor operations in the order determined in point (b) (only do one round of message passing for now). Guard against numerical over-/underflow by normalising each message after its calculation.
- (f) (In code) Now repeat the message-passing rounds of point (e) until inference has converged. Although there are many possible ways to check for convergence, I suggest that you calculate the *sepset* beliefs after each round of message passing, and calculate the change in *sepset* belief from the previous round for each *sepset*. To calculate the change in belief, I suggest you use the Kullback-Leibler divergence¹ (but there are other ways). If the change in beliefs for *all* the *sepsets* fall below a certain threshold (e.g. 10^{-6}), then you can consider inference to have converged. In `emdw`, you can calculate the KL-divergence between two `Factor` objects by calling the `distance` function². It might be interesting to display (or plot) the convergence behaviour as the biggest change in *sepset* belief as a function of the iteration (round) number.
- (g) (In code) Check that the CG is *calibrated* in the same way as in Question 1.1(e)
- (h) (In code) Extract the marginal beliefs over each of the transmitted bits, $b_0, \dots, b_6$ (using appropriate *sepset* or cluster beliefs). Compare these beliefs to that obtained in Question 1.1(f). Is BP in a loopy CG therefore *exact* or *approximate* inference? Although the beliefs might not be exact, can you still recover the correct transmitted sequence from the beliefs?
- (i) (On paper) For the Hamming (7,4) error-correction code, can you see any advantage to using a loopy CG instead of a JT?

1.3 Belief propagation (BP) using `emdw`'s built-in functionality

The goal of this question is to repeat Questions 1.1 and 1.2 (i.e., perform BP for a JT and a loopy CG), but instead use `emdw`'s built-in functionality to construct the CG and perform message passing for us.

In addition to the description of `emdw`'s built-in functionality given in Question 1.3 of Week 4's practical assignment, you will also need the following information.

More information about `emdw`'s built-in functionality:

- `emdw` creates a CG from the given factors and observations when you create a `ClusterGraph` object. You can specify the type of CG by passing a flag to the `ClusterGraph` constructor. In Week 4's practical, you created a factor graph (FG) by using the `ClusterGraph::BETHE` flag³. If you want to create a JT, then you should use the `ClusterGraph::JTREE` flag, and if you want to create an LTRIP⁴ CG, you should use the `ClusterGraph::LTRIP` flag.
- After constructing a `ClusterGraph` object `cg`, you can generate a `graphviz` .dot file to visualise the CG structure by using the following command::

¹You can think of the KL-divergence as the "distance" or "difference" between two distributions.

²For example, if `ptrX1` and `ptrX2` point to valid (normalised) distributions, then the KL-divergence between them is calculated by writing `ptrX1->distance(ptrX2)`.

³Remember that a FG is also called a *Bethe cluster graph*.

⁴The LTRIP algorithm to construct a CG is described in the paper by Simon Streicher and Johan du Preez, *Graph coloring: comparing cluster graphs to factor graphs*, in Proceedings of SAWACMMM'17, Mountain View, CA, USA, October 23, 2017, <https://doi.org/10.1145/3132711.3132717>. It can be used to construct a certain type of CG which often works well for real-world problems. However, you do not need to know how the LTRIP algorithm works.

```
cg.exportToGraphViz("your_name_for_the_graph");
```

When executing this function, the dot command to convert the .dot file to a .pdf file will be written to the screen. After locating the .dot file, apply this command. For example, to take the .dot file `graph.dot` and generate the .pdf file `graph.pdf`, type the following command in the terminal:

```
dot -Tpdf graph.dot > graph.pdf
```

- (a) (In code) Follow a similar procedure than that of Question 1.3 of Week 4 to construct a JT as a `ClusterGraph` object: Construct all the factors of the model *without* any evidence inserted, and add them to a `vector` container. Create a `map` container to hold the evidence, and add all the observed RV values, $r_0, \dots, r_6$, to this map. Then construct a `ClusterGraph` object with the `ClusterGraph::JTREE` flag, the `vector` of factors, and `map` of observations passed to the constructor as arguments.
- (b) (In code) Save the CG as a `graphviz` .dot file, convert it to a .pdf file, and inspect the .pdf file. Compare the JT created by `emdw` to that which you created by hand in Question 1.1(a).
- (c) (In code) Use `emdw`'s built-in functionality to perform message passing, and then extract the marginal beliefs over each of the transmitted bits, $b_0, \dots, b_6$. Compare these beliefs with that which you obtained in Question 1.1(f).
- (d) (In code) Repeat point (a), but now construct an LTRIP CG instead of a JT (by using the `ClusterGraph::LTRIP` flag). Repeat point (b) to save the CG as a .pdf file. Compare the loopy CG that `emdw` constructed with the one you constructed by hand in Question 1.2(a).
- (e) (In code) Use `emdw` to perform message passing and extract the marginal beliefs over each of the transmitted bits. Compare these beliefs with the ones you calculated manually in Question 1.2(h). Are the beliefs the same? If not, can you explain why?

2 The image denoising problem revisited

This question revisits the image denoising problem that we used in Question 2 of Week 3's practical. You can reuse and build on the work you did for Question 2 of Week 3's practical.

2.1 Belief propagation (BP) using `emdw`'s built-in functionality

The goal of this question is to use `emdw`'s built-in functionality to perform inference for both the small (4×4) and large (28×28) image.

- (a) (On paper) For a MN corresponding to a 2×2 image (and assuming that the pixels of the corrupted image are observed), go through the first part of the JT construction process: choose a good VE ordering, eliminate the RVs according to your chosen ordering, draw the induced MN, and determine the size of the largest maximal clique. The size of the maximal clique in the induced MN will be the size of the largest cluster in the JT, which is called the *tree width* of the JT. Now repeat this process to determine the tree width of a JT for a 3×3 image. Can you see a pattern in how the JT tree width relates to the dimensions of the image? If this pattern holds, what would be the tree width of a 4×4 image? And what would be the tree width of a 28×28 image? By calculating the number of entries in the table required for the largest cluster in the JT for the 4×4 image, do you think it would be practically possible to use a JT for this image size? And what about the 28×28 image?
- (b) (In code) Use `emdw`'s built-in functionality to construct a JT for the small (4×4) image, using the same parameter values as for Question 2 in Week 3. Then perform message passing, extract the marginal belief $p(x_i = 1 | \text{the distorted image})$ for each pixel, and write the results to a text file. Visualise the results (using e.g. Python) and compare them with that of Question 2 of Week 3.
- (c) (In code) Repeat point (b), but use a loopy CG by requesting `emdw` to construct an LTRIP CG. Compare the results with that of point (b).

- (d) (In code) From point (a), you should have concluded that it would not be practically possible to use a JT for the 28×28 image. So use a loopy CG to reconstruct the 28×28 image, using a similar approach to point (c). Visualise the reconstructed image, compare it with the ground truth image, and adjust the parameters until you get acceptable results. Look closely at the different regions of the reconstructed image: Can you explain why the reconstruction works well in certain places? And can you explain why the reconstruction is bad in other places?

2.2 Expanding the model for image denoising problem (optional)

For the image denoising problem so far, we have only used factors over neighbourhoods of two pixels (left-right or top-bottom) – this is depicted on the left of Figure 3. However, it is possible to use factors over larger

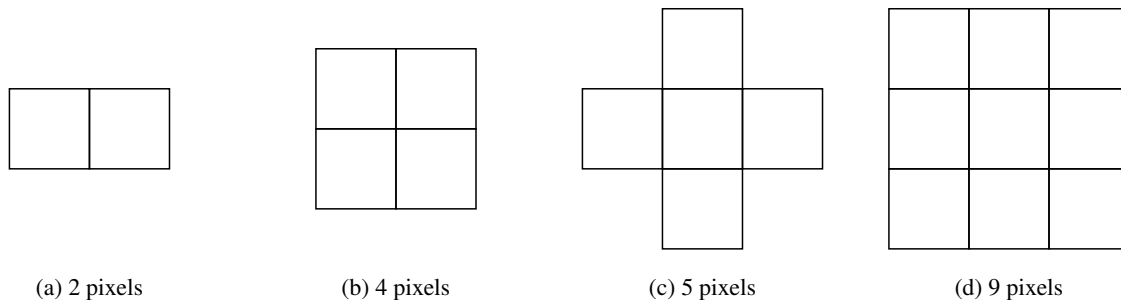


Figure 3: Example choices for neighbourhoods of pixels

neighbourhoods, as the other diagrams in Figure 3 illustrate. With larger neighbourhoods, it is possible to capture more complex relationships between pixels, possibly leading to much better reconstruction results.

For this optional question, you are encouraged to explore the use of factors over larger neighbourhoods. The idea is to expand your definition for the factors of the model to use larger neighbourhoods, and then to repeat Question 2.1(d).

3 Evaluation: Write and submit a short report

Document the following in a short report:

- Present the design and implementation of your inference procedure, the experimental results and the conclusions for Question 1.2 [4 marks]
- Present an overview of your implementation, the experimental results and conclusions for Question 1.3(d) and (e) [2 marks]
- Present your model design, overview of implementation, experimental results and conclusions for Question 2.1(d) [4 marks]
- (Optional) Present your model design, overview of implementation, experimental results and conclusions for Question 2.2 [10 bonus marks]

Submit both the report (in PDF format) and all your code for this assignment on STEMLearn by **Sunday 15 March at 23:59**.

See the DE424 info sheet for general guidelines for the practical report. Also see the document about how to avoid academic misconduct in DE424 on STEMLearn. Specific instructions:

- The text in the report should be typed, the page limit is **5 pages**⁵, and the report must be submitted in PDF format.

⁵This page limit is applicable to the compulsory part of the report. If you choose to do the optional question, then *the part of your report that presents your work on the optional question* may exceed the page limit.

- At the top of the report, you should include a declaration that the report and the work presented in it are your own work. If you received any help or information from any source that contributed to the work or report, you should also mention the source and the extent of the help or information in the declaration.
- The report should be clear, but you do not have to spend much time on making it beautiful. You can, for instance, draw diagrams or write mathematical manipulations by hand, scan it in, and include it in your report as an image (instead of drawing diagrams in applications such as Inkscape or typing the mathematical manipulations as equations in $\text{\LaTeX}$).
- You should give enough context in your report so that someone who is familiar with the module can understand how you have solved the problem and how well it worked without having to look at your code. Make sure that all your design decisions are clearly documented (and – if necessary – motivated), for example:
 - your message-passing schedule
 - how you tested for convergence
 - your choice of parameter values, and how you have chosen them

However, you should also be concise: avoid giving lengthy discussions of obvious matters, showing minute steps in mathematical manipulations, or copying large parts of your code into your report.